Evaluating Clustering Algorithms and LLM Prompting Strategies for Architecture Recovery in Apache Lucene Codecs

Christopher Seibel Alexander Makejkin Louis Husemann Mikhail Kiselev
Mohammed Danish Shaikh

May 15, 2026

1 Introduction

As software systems evolve, their architectural documentation frequently deviates from the actual implementation. To maintain system integrity and facilitate comprehension, automated architecture recovery techniques are essential. This study evaluates the efficacy of structural clustering algorithms and Large Language Models (LLMs) in extracting and describing the architectural components of an existing software system, specifically focusing on the **Codecs** component of the **Apache Lucene** project.

To address the first research question (**RQ1**), the **ARCADE** tool suite is utilized to extract system-wide dependencies and isolate component-relevant relationships. Three distinct structural clustering algorithms are applied to the filtered dependency graph:

- **Weighted Combined Algorithm (WCA):** A hierarchical clustering approach utilizing edge weights to determine cluster merges.
- **LIMBO:** An information-theoretic algorithm designed to minimize the loss of mutual information during clustering.
- **ACDC:** A heuristic-based algorithm that identifies architectural patterns based on structural connectivity.
- **ARC (Semantic-Structural Clustering):** A hybrid approach operating at the file level. It fuses a normalized structural dependency graph with a semantic similarity matrix, which is computed via high-dimensional vector embeddings generated by the `Qodo/Qodo-Embed-1-7B` model.

The structural accuracy of these algorithms is quantified using Architecture-to-Architecture (**a2a**) and Cluster Coverage (**cvg**) metrics. Technical implementation details and CLI commands are provided in Appendix A.
To address the second research question (**RQ2**), this study investigates the intersection of structural recovery and generative AI by evaluating how different prompting techniques influence an LLM's ability to synthesize high-level architectural descriptions from source code. This evaluation implements a dual-stage pipeline:

- **Prototyping Tier:** Local evaluation utilizing the lightweight `Mistral-7B-Instruct-v0.3` model within Google Colab. This phase leverages the model's native function calling and specialized tokenizer to verify prompt logic stability on structural data.
- **Production Tier:** Scaled execution utilizing the heavyweight `Mistral-Large-3-675B-Instruct-2512` model deployed on a High-Performance Computing (HPC) cluster via Slurm. This phase processes the complete Codecs component to generate concise architectural titles and high-level descriptive summaries for each identified cluster.

Ultimately, this work aims to quantify the variance in accuracy across different clustering algorithms and determine the impact of prompting strategies on the semantic integrity of AI-generated architectural summaries.

2 Study Design

2.1 Research Questions

- **RQ1:** How do different clustering algorithms vary in their ability to accurately determine the architectural components of a system?
- **RQ2:** How do different prompting techniques impact LLM's ability to accurately describe architectural components based on source code?

2.2 Methodology

The experimental setup follows a phased workflow designed to recover the architecture of the Apache Lucene Codecs component through a combination of structural clustering and Large Language Model (LLM) analysis.

2.2.1 Dependency Extraction and Preprocessing

The initial phase involves compiling the Java source code to generate compiled *.jar files. System-wide dependencies are extracted using the ARCADE JavaParser tool, which produces a master .rsf file representing all project relationships. To isolate the scope to the *Codecs* component, the dependencies are filtered such that only relationships where the source or target packages reside within the component's namespace are retained.

2.2.2 Semantic-Structural ARC Clustering

To fuse structural dependencies with semantic source code relationships, the Agglomerative Recovery of Components (ARC) methodology is applied. This involves a multi-step feature extraction and matrix fusion process:

- **Semantic Embedding Generation:** For each valid source code file within the target component, semantic vectors are extracted using the `Qodo/Qodo-Embed-1-7B` model. To ensure hardware efficiency, the model is executed using 4-bit quantization.
- **Similarity Matrices:** A semantic similarity matrix is constructed by computing the cosine similarity between the generated file embeddings. In parallel, a normalized structural similarity matrix is derived from the filtered `.rsf` dependency graph, calculating the relative package dependency overlap between file pairs.
- **Matrix Fusion and Clustering:** Both matrices are linearly combined using a weighting parameter α, where α dictates the proportion of structural versus semantic data. The resulting similarity matrix is inverted into a distance matrix and processed via *Agglomerative Clustering* (utilizing both *average* and *complete* linkage strategies) to generate the final file-level architecture components.

2.2.3 LLM Prompting Strategy (RQ2)

Following the architectural extraction, an LLM-based prompting workflow is utilized to generate architectural descriptors for the identified clusters.

Prototyping Tier: Local Validation

- **Environment:** Prototyping is executed within Google Colab to verify prompt logic stability on smaller samples before full deployment.
- **Technical Capabilities:** The model's native support for function calling and the specialized `Mistral-7B-Instruct-v0.3` tokenizer are leveraged to handle structural architectural data efficiently.

Production Tier: Heavyweight Scaling Upon validation of the prompting logic, the recovery process is scaled to the university's High Performance Computing (HPC) cluster.

- **Model Scaling:** The study utilizes the heavyweight – `Mistral-Large-3-675B-Instruct-2512` to process the complete Codecs component of the Lucene project.
- **Execution Framework:** Scaling is managed via Slurm job scheduling.
- **Output Generation:** The primary goal of this tier is to generate concise architectural titles and a high-level descriptive summary for each cluster identified in the structural recovery phase.

2.3 Data Analysis Process

The data analysis pipeline is structured into three primary phases: structural normalization, quantitative statistical profiling, and comparative similarity evaluation.

2.3.1 Structural Normalization and Metadata Extraction

Raw architectural outputs from the ARCADE toolset are generated in the *.rsf* format, which represents dependencies line-by-line as `depends Source_Package Target_Package`. To facilitate objective comparison, a post-processing utility is utilized to transform these raw files into structured plain-text artifacts. This utility performs the following operations:

- **Sorting:** Clusters are ordered by cardinality (size), and internal elements are sorted alphabetically to ensure deterministic comparison.
- **Metadata Generation:** For each algorithmic run, the utility extracts a statistical profile used to determine the optimality of the clustering parameters.

Resolution of Granularity Discrepancies A critical methodological challenge arose from the varying input granularity requirements of the applied clustering approaches. The heuristic baselines established in Week 1 (WCA, LIMBO, ACDC) operated on class-level dependencies, inherently preserving inner-class relationships Conversely, the ARC clustering and the underlying semantic embedding models (RQ2) process source code strictly at the file level (`.java`). To establish mathematically valid comparability for the *a2a* and *cvg* metrics, a deterministic reduction was applied. All inner-class definitions within the original `filtered.rsf` were merged into their respective parent file structures, producing a flattened `filtered-without-innerclasses.rsf`. The structural baselines (WCA, LIMBO, ACDC) were subsequently re-executed on this normalized dependency graph to ensure isomorphic entity sets across all compared architectures. [I dont know if its supposed to be here Mr Tutor, question for you if we should move this down to the threats to validity maybe?]

2.3.2 Quantitative Profiling Metrics

Algorithm selection and parameter tuning are based on a set of distribution metrics that characterize the resulting architecture. These metrics include:

- **Cluster Cardinality:** Total number of identified clusters and the count of singletons (size $= 1$) versus multi-element clusters (size > 1).
- **Central Tendency:** Average size per cluster across the entire system and specifically for clusters with size > 1.
- **Distribution Variance:** Median size for multi-element clusters and the absolute size of the largest identified cluster.

2.3.3 Comparative Similarity Evaluation (RQ1)

To quantify the variance between clustering algorithms, the processed *.rsf* files are analyzed using the **Architecture-to-Architecture** and **Cluster Coverage** metrics.

- **a2a Similarity:** Measures the global structural distance between two architectures, providing a percentage-based similarity score.
- **Cluster Coverage (cvg):** An asymmetric metric measuring the extent to which clusters in one architecture overlap with clusters in another.

A high percentage in these metrics indicates a high degree of structural convergence between different algorithmic approaches.

2.3.4 Qualitative LLM Assessment (RQ2)

For the evaluation of LLM-generated architectural descriptions, a qualitative assessment is conducted. This involves mapping the generated titles and high-level summaries back to the source code elements within each cluster to evaluate semantic accuracy and the impact of different prompting techniques.
TODO in further weeks.

3 Results

This section presents the quantitative findings derived from the structural clustering of the Apache Lucene Codecs component.

3.1 Algorithmic Profiling and Cluster Distribution (RQ1)

The statistical profiles of the generated architectures for WCA and LIMBO are summarized in Table 1 and 2.

Table 1: Quantitative Profiling of Clustering Algorithms (Targets: 70 and 50 Clusters)

	Target: 70 Clusters			**Target: 50 Clusters**		
	WCA (UEMNM)	WCA (UEM)	LIMBO	WCA (UEMNM)	WCA (UEM)	LIMBO
Cluster Cardinality						
Total number of clusters	70	70	70	50	50	50
Number of clusters (size = 1)	48	45	26	41	41	12
Number of clusters (size > 1)	22	25	44	9	9	38
Central Tendency						
Avg. size per cluster	2.70	2.70	2.70	3.78	3.78	3.78
Avg. size (clusters > 1)	6.41	5.76	3.70	16.44	16.44	4.66
Distribution Variance						
Median size (clusters > 1)	6	6	2	16	13	4
Largest cluster size	11	10	41	28	42	41

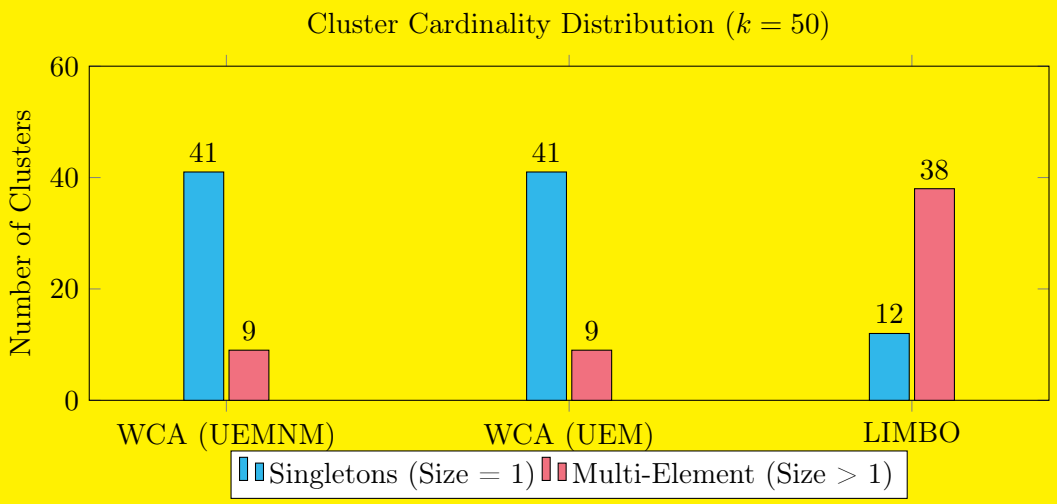

Figure 1: Comparison of cluster composition at a target granularity of $k = 50$. WCA primarily isolates singletons, whereas LIMBO achieves a broader distribution of multi-element clusters.

Table 2: Quantitative Profiling of the Clustering Algorithm ACDC

	ACDC
Cluster Cardinality	
Total number of clusters	5
Number of clusters (size = 1)	0
Number of clusters (size > 1)	5
Central Tendency	
Avg. size per cluster	37.8
Avg. size (clusters > 1)	37.8
Distribution Variance	
Median size (clusters > 1)	26
Largest cluster size	97

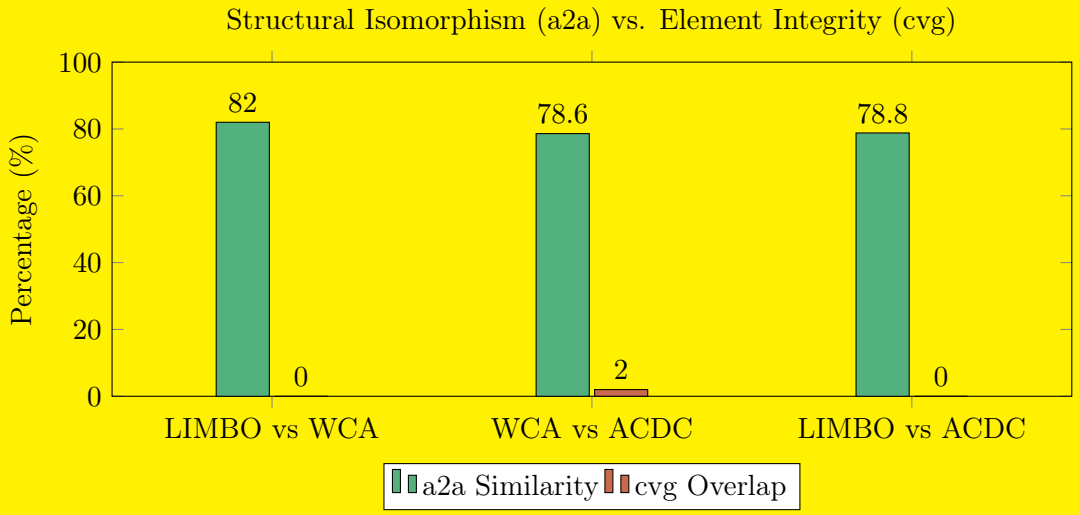

Figure 2: Visualization of the metric discrepancy: Despite high topological congruence (a2a $\approx$ 80%), the algorithms fail to converge on the concrete allocation of elements (cvg $\approx$ 0%).

3.1.1 Structural Similarity Evaluation

Table 3 illustrates the structural similarity and overlap across the algorithm pairs. To provide a comprehensive comparison, the WCA algorithm is explicitly differentiated by its underlying similarity measures: UEM and UEMNM. All evaluations were conducted on architectures partitioned into exactly 50 clusters.

Table 3: Architecture-to-Architecture (a2a) and Cluster Overlap (cvg)

Cluster Size 50	a2a Similarity	cvg (A $\rightarrow$ B)	cvg (B $\rightarrow$ A)
LIMBO vs WCA (UEM)	82.02%	0.0%	0.0%
LIMBO vs WCA (UEMNM)	82.23%	0.0%	0.0%
WCA (UEM) vs ACDC	78.59%	2.0%	5.26%
WCA (UEMNM) vs ACDC	78.15%	2.0%	5.26%
LIMBO vs ACDC	78.81%	0.0%	0.0%

3.1.2 Semantic-Structural ARC Clustering Evaluation

To further evaluate architecture recovery performance, we applied ARC clustering, which combines both structural dependencies and semantic similarity derived from source code embeddings. ARC operates at file level rather than class level. Therefore, the baseline WCA, LIMBO, and ACDC architectures were regenerated using a filtered dependency graph without inner classes to ensure comparability. Multiple ARC configurations were evaluated by adjusting the weighting parameter *alpha*, linkage strategy, and target cluster count. The parameter *alpha* controls the contribution of structural versus semantic similarity within the combined similarity matrix.

Structural Similarity Evaluation: $k = 50$ vs. $k = 57$ In this evaluation, the LLM-generated architectures (QODO) were tested against established baseline heuristics (ACDC, LIMBO, WCA). Regardless of the specific hyperparameter configuration—including the structural-semantic weight ($\alpha \in [0.25, 0.75]$) and the linkage strategy (e.g., average, complete)—all QODO-based ARC variants exhibit structural isomorphism at a fixed target cluster count (k). Therefore, their results are presented in an aggregated format. Table 4 contrasts the outcomes for cluster sizes 50 and 57.

Table 4: Comparison of Similarity Metrics (a2a) and Overlap (cvg)

Cluster Size	Algorithm Pair	a2a Similarity	cvg (A → B)	cvg (B → A)
$k = 50$	ARC vs LIMBO	78.33%	0.80%	0.80%
	ARC vs WCA (UEM)	70.20%	0.10%	0.10%
	ARC vs WCA (UEMNM)	70.69%	0.10%	0.10%
	ARC vs ACDC	61.33%	0.00%	0.00%
$k = 57$	ARC vs LIMBO	80.71%	0.93%	0.93%
	ARC vs WCA (UEM)	72.14%	0.09%	0.09%
	ARC vs WCA (UEMNM)	72.14%	0.11%	0.11%
	ARC vs ACDC	59.69%	0.00%	0.00%

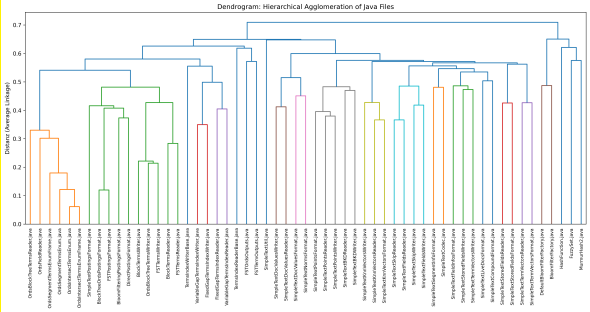

Figure 3: ARC Clustering: $\alpha = 0.50$, Average Linkage

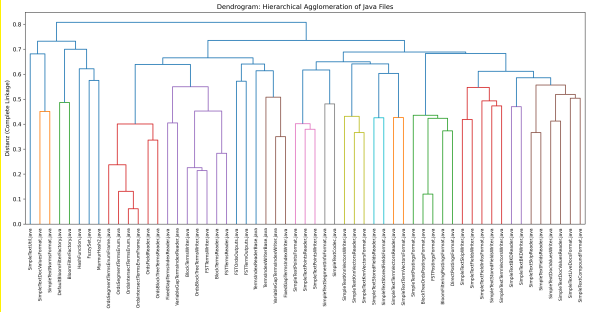

Figure 4: ARC Clustering: $\alpha = 0.50$, Complete Linkage

[DECLARATION LLM GENERATED SINCE I HAVENT GOTTEN BEHIND HOW TO INTERPRET THESE GRAPHS, and ignore that they are so small ! will be reworked ![]The comparison reveals significant topological variances: Average Linkage tends to form highly asymmetric, chain-like structures (chaining effect), whereas Complete Linkage enforces more compact and balanced cluster subgraphs.

4 Discussion

This section interprets the quantitative findings from the structural recovery phase and evaluates the efficacy of the applied clustering algorithms in accurately determining the architectural components of the Apache Lucene Codecs module.

The ARC clustering evaluation demonstrated that parameter selection significantly influenced architectural similarity. Adjusting only the semantic weighting parameter *alpha* produced small variations in similarity scores. However, modifying the clustering configuration through complete linkage and increasing the target number of clusters from 10 to 30 resulted in a much higher a2a similarity across all baseline algorithms. The strongest similarity was observed between ARC and ACDC, reaching 67.97%, thus suggesting that ARC clustering aligns well with heuristic architectural decomposition produced by ACDC.

4.1 Interpretation of Clustering Profiles (RQ1)

An analysis of the clustering profiles indicates distinct algorithmic behaviors in architecture recovery. The pattern-based algorithm ACDC (Table 2) acts as a heuristic baseline. By dynamically resolving to 5 sub components without producing any singleton clusters, ACDC demonstrates

aggressive orphan adoption and subgraph dominance. This results in highly abstract, large-scale subsystems with a median size of 26 and maximum 97, rather than fine-grained modules.

In contrast WCA and LIMBO exhibit strong sensitivity to the target granularity (k). At a fine-grained target of $k = 70$ (Table 1), WCA (particularly with UEMNM) yields a structurally cohesive distribution. It produces a higher median cluster size (6) and a bounded maximum cluster size (11), successfully isolating moderately sized, functional modules. Conversely, LIMBO demonstrates an unbalanced bottom-up aggregation: it forms a massive monolithic cluster (size 41) containing structurally disparate elements, while fragmenting the remainder into a high volume of micro-clusters (median size 2).

Reducing the target granularity to $k = 50$ (Table 1) exacerbates algorithmic biases and alters the topology significantly. WCA isolates an extreme number of singleton classes (41 out of 50 clusters are of size 1), forcing the remaining system entities into merely 9 dense clusters (maximum size up to 42 for UEM). This indicates that WCA struggles to establish mid-level cohesion at this abstraction level, instead isolating outliers and heavily clumping the core architecture. LIMBO, while maintaining its primary monolithic cluster (which remains static at size 41), provides a more distributed secondary architecture, grouping the remaining entities across 38 multi-element clusters with a more stable median size of 4.

4.1.1 Analysis of Structural Similarity and Overlap

As detailed in Table 3, the Architecture-to-Architecture (a2a) evaluation reveals the highest macro-structural similarity (up to 82.23%) between LIMBO and the WCA variants. ACDC exhibits a marginally lower, yet consistent, similarity score (∼78.5%) when compared to both hierarchical approaches.

However, the global structural similarity masks severe discrepancies in specific element allocation, which are starkly exposed by the Cluster Coverage (cvg) metric. Across almost all baseline comparisons, the overlap registers at exactly 0.0%. Most notably, the coverage between LIMBO and WCA, as well as LIMBO and ACDC, indicates a complete lack of identical clusters. While the algorithms construct analogous macro-topologies (similar cluster size distributions), they populate these clusters with entirely disjointed entity sets.

The only measurable, albeit marginal, overlap exists between WCA and ACDC (2.0% vs. 5.26%). The inherent asymmetry of this cvg variance is heavily influenced by the differing total cluster counts (fixed $k = 50$ for WCA vs. dynamic $k = 5$ for ACDC). Overall, the near-zero coverage across all structural baselines highlights that file-level dependency analysis (without inner classes) creates highly volatile partitioning landscapes. Heuristic (ACDC), information-theoretic (LIMBO), and edge-weight-based (WCA) algorithms fail to converge on identical micro-architectures, despite operating on the exact same normalized `.rsf` graph.

4.2 Semantic vs. Structural Divergence

The comparison of semantic clustering outputs against structural baselines yields critical insights regarding the model's architecture recovery performance:

- **Scaling Effects ($k = 50 \rightarrow k = 57$):** For LIMBO and WCA, a higher cluster granularity ($k = 57$) correlates with an increase in *a2a Similarity* by approximately 2 percentage points. This indicates that the LLM-generated partitioning converges more closely with the structural heuristics at a finer granularity. Conversely, the similarity to ACDC decreases slightly, attributable to the unpredictability of ACDC's dynamic partitioning when evaluated against fixed cluster constraints.

- **Structural Isomorphism vs. Element Integrity:** The most prominent result is the significant discrepancy between high *a2a* ($> 70\%$) and marginal *cvg* ($< 1\%$).

- The **a2a Similarity** indicates that the topology and relative distribution of the clusters are highly consistent with LIMBO/WCA. The LLM's system comprehension regarding functional groupings aligns with the baseline algorithms on a macro-architectural level.
- However, the **cvg (Coverage)** reveals that the exact allocation of individual entities (classes/files) diverges almost completely. While the algorithms construct similar "containers" (clusters), they populate them with entirely different elements.

- **Methodological Cause:** The baseline algorithms (particularly LIMBO and WCA) operate primarily on *Structural Dependencies* (e.g., method calls, inheritance). In contrast, the LLM (QODO) utilizes *Semantic Embeddings* (e.g., identifiers, comments, natural language semantics). The results demonstrate that while semantic coherence and structural coupling generate analogous abstraction levels (cluster count/size distribution) within this dataset, they act orthogonally concerning the concrete allocation of elements.

4.3 Implications for Researchers

[What theoretical insights were gained? How does this advance the state of the art?]

4.4 Implications for Practitioners

[Concrete recommendations for action for software engineers, architects, or management based on the quantitative metrics.]

5 Threats to Validity

[Critical reflection on the study design.]

- **Internal Validity:** An identified bug in the ARCADE Clusterer prevents the correct functionality of the `stopthreshold` argument. Consequently, the algorithms default to generating at least 50 clusters, limiting comparability for smaller target cluster counts.
- **Construct Validity:** *[Do the chosen metrics actually measure what they are intended to?]*
- **External Validity:** *[Are the results generalizable to other datasets?]*
- **Dependency Scope and Entity Truncation:** During the extraction of semantic embeddings, an ambiguity in resolving target dependencies from the `.rsf` file was identified. While the filtered dependency graph referenced 98 distinct entities across the global `lucene/lucene/` directory, only 57 files were strictly localized within the assigned `codecs` component focus area. To prevent embedding noise and mitigate cross-directory naming collisions, the semantic analysis scope was rigidly bounded to the `codecs` package. Consequently, out-of-package dependencies—while structurally present in the graph—were semantically truncated. This localized extraction potentially skewed the combined similarity matrix in the ARC clustering setup, artificially inflating the structural weight of missing external semantics.

6 Conclusion and Future Work

[Compact synthesis of the work. Identification of technical or conceptual limitations. Outline concrete next steps for future work.]

7 Hours spent per student

Name	W1	W2	W3	W4	W5
Christopher Seibel	1	4	8	0	0
Alexander Makejkin	4	2	3.5	0	0
Louis Husemann	3	3	7	0	0
Mikhail Kiselev	3	3	3	0	0
Mohammed Danish Shaikh	3	3	4	0	0

A Commands and Technical Reference

A.1 Cluster Generation

WCA:

```
java -Xmx4096m -jar arcade_core_clusterer.jar language=java \
deps=filtered.rsf projname=luccodecs projversion=9.0.0 \
projpath=output-cluster packageprefix="org.apache.lucene.codecs" \
algo=WCA measure=<UEMNM or UEM> \
serial=STEPCOUNT serialthreshold=1
```

LIMBO:

```
java -Xmx4096m -jar arcade_core_clusterer.jar language=java \
deps=filtered.rsf projname=luccodecs projversion=9.0.0 \
projpath=output-cluster packageprefix="org.apache.lucene.codecs" \
algo=LIMBO measure=IL \
serial=STEPCOUNT serialthreshold=1
```

ACDC:

```
java -jar arcade_core-ACDC.jar filtered.rsf acdc.rsf
```

A.2 Metrics Evaluation

a2a & cvg :

```
java -jar arcade_core_A2a.jar ./file1.rsf ./file2.rsf
java -jar arcade_core_Cvg.jar ./file1.rsf ./file2.rsf
```